Data Warehouse Complet — Projet Airline Analytics

Date : 2026-04-24 | **École :** Esprit — 3ème année

---

1. Compréhension du besoin

Contexte métier

Compagnie aérienne canadienne cherchant à améliorer deux axes stratégiques :

- **Axe Satisfaction** : Passer de 57% à 73.1% de passagers satisfaits
- **Axe Loyalty** : Augmenter le taux de fidélisation de 30% à 40%

Questions métier que le DWH doit répondre

#	Question Métier	Source	KPI lié
1	Quels services génèrent le plus d'insatisfaction ?	Satisfaction CSV	Avg Score par service
2	Quel est le taux de churn par segment (carte, profil) ?	Loyalty History CSV	Churn Rate (%)
3	Quels profils clients volent le plus et accumulent le plus de points ?	Flight Activity CSV	Avg Flights, Avg Points
4	Y a-t-il corrélation entre CLV et type de carte ?	Loyalty History CSV	Avg CLV par carte
5	Comment évolue la satisfaction selon la classe de voyage ?	Satisfaction CSV	Satisfaction Rate par classe
6	Quels mois/trimestres voient le plus de désabonnements ?	Calendar + Loyalty	Cancellations par trimestre

KPIs cibles

```
KPI Satisfaction :
  - Satisfaction Rate (%) = Satisfied / Total × 100        → cible : 73.1%
  - Avg Score by Service (1-5)                              → identifier les < 3.5
  - NPS Proxy = Satisfied - Dissatisfied                    → valeur positive
  - Delay Impact Score (satisfaction par tranche de retard)

KPI Loyalty :
  - Churn Rate (%)        = Cancellations / Total Members  → cible : ≤ 60%
  - Retention Rate (%)    = 1 - Churn Rate                 → cible : ≥ 40%
  - Avg Points Accumulated per Customer
  - Avg CLV by Card Type
  - Points Redemption Rate = Points Redeemed / Points Accumulated
```

---

2. Analyse des données sources

Vue d'ensemble des fichiers

Fichier	Lignes	Colonnes	Encodage	Destination
`airline_passenger_satisfaction.csv`	~129 880	24	UTF-8	`stg_airline_satisfaction` → `FACT_PASSENGER_SATISFACTION`
`Customer Loyalty History.csv`	~16 737	16	UTF-8 BOM	`stg_customer_loyalty` → `DIM_CUSTOMER`, `DIM_GEOGRAPHY`, `DIM_LOYALTY_CARD`
`Customer Flight Activity.csv`	~392 936	8	UTF-8 BOM	`stg_flight_activity` → `FACT_FLIGHT_ACTIVITY`
`Calendar.csv`	2 557	4	UTF-8	`stg_date` → `DIM_DATE`

Problèmes identifiés et solutions

Fichier	Problème	Solution SSIS/SQL
Satisfaction	`Arrival Delay` contient des NULLs	`TRY_CAST(NULLIF(...,'') AS INT)` → NULL conservé
Satisfaction	Aucun champ date	Limite documentée — SK_Date=-1 (Unknown Member)
Satisfaction	Aucun Loyalty Number	Deux visions indépendantes — pas de lien inter-datasets
Satisfaction	Scores `0` = service non applicable	Conservés (documentés) — exclus des moyennes avec `WHERE score > 0`
Loyalty	`Salary` — valeurs négatives, nombreux NULLs	Non chargé dans DWH (hors périmètre analytique)
Loyalty	`CLV` stocké en NVARCHAR dans raw_	`TRY_CAST(NULLIF(CLV,'') AS DECIMAL(18,2))`
Loyalty	`Cancellation Year/Month` vides = membres actifs	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)` → NULL
Activity	`Total Flights = 0`	Conservé (mois inactifs = information utile)
Calendar	Données propres	TRY_CAST direct sur champs DATE

Lien inter-datasets : Les datasets Satisfaction et Loyalty ne partagent **aucune clé commune**. Limite connue et documentée — deux visions complémentaires du métier.

---

3. Architecture ETL (SSIS — SQL Server Integration Services)

Approche : 100% SSIS + T-SQL inline

Toutes les opérations (extraction, transformation, chargement) sont réalisées dans des **Execute SQL Tasks** au sein de packages SSIS. Aucun outil externe (Python, PowerShell, etc.) n'est utilisé.

Architecture 5 packages + 1 master

```
Master.dtsx  (orchestrateur)
  │
  ├─► 00_Create_Database.dtsx   — Initialisation des bases de données
  │
  ├─► 01_Load_Staging.dtsx      — Chargement et nettoyage vers STAGING_DB
  │
  ├─► 02_Populate_Dimensions.dtsx — Alimentation des 5 tables DIM dans DWH_DB
  │
  ├─► 03_Populate_Facts.dtsx    — Alimentation des 2 tables FACT dans DWH_DB
  │
  └─► 04_Finalize.dtsx          — Index, statistiques, validation finale
```

Flux de données global

```
4 fichiers CSV
      │
      ▼ [BULK INSERT — CODEPAGE 65001, FIELDQUOTE='"', FIRSTROW=2]
4 tables raw_ (tout NVARCHAR)     → STAGING_DB
      │
      ▼ [Execute SQL Task — TRY_CAST, NULLIF, ISNULL, calculs dérivés]
4 tables stg_  (types corrects)   → STAGING_DB
      │
      ▼ [Execute SQL Task — SELECT DISTINCT + NOT EXISTS guards]
5 tables DIM_                     → DWH_DB
      │
```

```
      ▼ [Execute SQL Task — Lookups SK via sous-requêtes corrélées]
2 tables FACT_                    → DWH_DB
      │
      ▼ [Index NC + UPDATE STATISTICS + validation des counts]
DWH opérationnel
```

---

4. Détail de chaque package SSIS

Package 00 — `00_Create_Database.dtsx`

Rôle : Initialisation complète des bases de données. Exécuté une seule fois au départ du pipeline.

Tâche	Action
`SQL_Reset_Staging`	Crée le schéma `etl` et la table `etl.run_log` ; supprime les tables `raw_*` et `stg_*` résiduelles ; recrée les 4 tables `stg_` avec types corrects
`SQL_Create_DWH_DB`	Crée `DWH_DB` si elle n'existe pas
`SQL_Create_DWH_Tables`	Crée les 5 tables DIM et 2 tables FACT avec leurs colonnes, types et contraintes PK
`SQL_Unknown_Members`	Insère SK=-1 dans chaque DIM (Unknown Member) via `SET IDENTITY_INSERT ON/OFF`

Tables stg_ créées (types corrects) :

```
-- stg_airline_satisfaction
ID INT, Gender NVARCHAR(50), Age INT, CustomerType NVARCHAR(50),
TypeOfTravel NVARCHAR(50), Class NVARCHAR(50), FlightDistance INT,
DepartureDelay INT NULL, ArrivalDelay INT NULL,
TimeConvenience INT, ...(14 scores)..., BaggageHandling INT,
AvgServiceScore DECIMAL(5,2), Satisfaction NVARCHAR(50), Satisfaction_Flag BIT

-- stg_flight_activity
LoyaltyNumber NVARCHAR(50), Year INT, Month INT, TotalFlights INT,
Distance DECIMAL(18,2), PointsAccumulated DECIMAL(18,2),
PointsRedeemed DECIMAL(18,2), DollarCostPointsRedeemed DECIMAL(18,2)

-- stg_customer_loyalty
LoyaltyNumber NVARCHAR(50), Country NVARCHAR(100), Province NVARCHAR(100),
City NVARCHAR(100), PostalCode NVARCHAR(20), Gender NVARCHAR(20),
Education NVARCHAR(100), Salary NVARCHAR(50), MaritalStatus NVARCHAR(50),
LoyaltyCard NVARCHAR(50), CLV DECIMAL(18,2), EnrollmentType NVARCHAR(50),
EnrollmentYear INT, EnrollmentMonth INT, CancellationYear INT NULL, CancellationMonth INT NULL

-- stg_date
DateValue DATE, StartOfYear DATE, StartOfQuarter DATE, StartOfMonth DATE
```

Package 01 — `01_Load_Staging.dtsx`

Rôle : Extraire les 4 CSV, les nettoyer et les charger dans les tables `stg_`.

Flux d'exécution (11 tâches) :

```
SQL_Prepare_Staging
        │
        ├──► SQL_BulkLoad_Satisfaction ──► SQL_Transform_Satisfaction ─┐
        ├──► SQL_BulkLoad_Activity     ──► SQL_Transform_Activity     ─┤
        ├──► SQL_BulkLoad_Customer     ──► SQL_Transform_Customer     ─┤──► SQL_Rebuild_Staging ──►
SQL_Validate_Staging
        └──► SQL_BulkLoad_Date         ──► SQL_Transform_Date         ─┘
```

```
                (4 BulkLoad en parallèle)      (4 Transform en parallèle)    (attend les 4 —
LogicalAnd=True)
```

Tâche 1 — `SQL_Prepare_Staging`

- Supprime les tables `raw_*` résiduelles d'une éventuelle exécution précédente
- Truncate les 4 tables `stg_*` (les vide sans les supprimer)
- Supprime les index NC résiduels
- Crée les 4 tables `stg_raw_*` avec **toutes les colonnes NVARCHAR** (zone d'atterrissage brute)

Tâches 2-5 — `SQL_BulkLoad_*` (en parallèle)

```
-- Paramètres communs à tous les BULK INSERT
FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='\n',
CODEPAGE='65001', FIELDQUOTE='"', TABLOCK
```

Tâche	Fichier source	Table cible
`SQL_BulkLoad_Satisfaction`	`airline_passenger_satisfaction.csv`	`stg_raw_satisfaction`
`SQL_BulkLoad_Activity`	`Customer Flight Activity.csv`	`stg_raw_flight_activity`
`SQL_BulkLoad_Customer`	`Customer Loyalty History.csv`	`stg_raw_customer_loyalty`
`SQL_BulkLoad_Date`	`Calendar.csv`	`stg_raw_date`

Tâche 6 — `SQL_Transform_Satisfaction`

Transformations appliquées (raw_ → stg_airline_satisfaction) :

Colonne source (raw_)	Transformation SQL	Colonne cible (stg_)	Type
`raw_ID`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`ID`	INT
`raw_Gender`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Gender`	NVARCHAR(50)
`raw_Age`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`Age`	INT
`raw_CustomerType`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`CustomerType`	NVARCHAR(50)
`raw_TypeOfTravel`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`TypeOfTravel`	NVARCHAR(50)
`raw_Class`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Class`	NVARCHAR(50)
`raw_FlightDistance`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`FlightDistance`	INT
`raw_DepartureDelay`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`DepartureDelay`	**INT NULL**
`raw_ArrivalDelay`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`ArrivalDelay`	**INT NULL**
`raw_TimeConvenience` … `raw_BaggageHandling`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)` ×14	`TimeConvenience` … `BaggageHandling`	INT ×14
(dérivé)	`CAST((col1+col2+…+col14) AS DECIMAL(5,2)) / 14.0`	`AvgServiceScore`	DECIMAL(5,2)
`raw_Satisfaction`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Satisfaction`	NVARCHAR(50)
(dérivé)	`CASE WHEN LOWER(LTRIM(RTRIM(raw_Satisfaction)))='satisfied' THEN 1 ELSE 0 END`	`Satisfaction_Flag`	BIT

Filtre : `WHERE NULLIF(LTRIM(RTRIM(raw_ID)),'') IS NOT NULL` (élimine les lignes d'entête résiduelles)

Tâche 7 — `SQL_Transform_Activity`

Colonne source (raw_)	Transformation SQL	Colonne cible (stg_)	Type
`raw_LoyaltyNumber`	`LTRIM(RTRIM(...))`	`LoyaltyNumber`	NVARCHAR(50)
`raw_Year`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`Year`	INT
`raw_Month`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`Month`	INT
`raw_TotalFlights`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`TotalFlights`	INT
`raw_Distance`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`Distance`	DECIMAL(18,2)
`raw_PointsAccumulated`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`PointsAccumulated`	DECIMAL(18,2)
`raw_PointsRedeemed`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`PointsRedeemed`	DECIMAL(18,2)
`raw_DollarCostPointsRedeemed`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`DollarCostPointsRedeemed`	DECIMAL(18,2)

Tâche 8 — `SQL_Transform_Customer`

Colonne source (raw_)	Transformation SQL	Colonne cible (stg_)	Type
`raw_LoyaltyNumber`	`LTRIM(RTRIM(...))`	`LoyaltyNumber`	NVARCHAR(50)
`raw_Country` … `raw_PostalCode`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Country` … `PostalCode`	NVARCHAR
`raw_Gender`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Gender`	NVARCHAR(20)
`raw_Education`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Education`	NVARCHAR(100)
`raw_Salary`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Salary`	NVARCHAR(50) — **non chargé en DWH**
`raw_MaritalStatus`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`MaritalStatus`	NVARCHAR(50)
`raw_LoyaltyCard`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`LoyaltyCard`	NVARCHAR(50)
`raw_CLV`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`CLV`	DECIMAL(18,2)
`raw_EnrollmentType`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`EnrollmentType`	NVARCHAR(50)
`raw_EnrollmentYear`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`EnrollmentYear`	INT
`raw_EnrollmentMonth`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`EnrollmentMonth`	INT
`raw_CancellationYear`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`CancellationYear`	**INT NULL**
`raw_CancellationMonth`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`CancellationMonth`	**INT NULL**

Tâche 9 — `SQL_Transform_Date`

Colonne source	Transformation	Colonne cible	Type
`raw_Date`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`DateValue`	DATE
`raw_StartOfYear`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`StartOfYear`	DATE
`raw_StartOfQuarter`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`StartOfQuarter`	DATE
`raw_StartOfMonth`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`StartOfMonth`	DATE

Filtre : `WHERE TRY_CAST(...raw_Date... AS DATE) IS NOT NULL`

Tâche 10 — `SQL_Rebuild_Staging`

- Supprime les 4 tables `raw_*` (zone d'atterrissage plus nécessaire)
- Crée 4 index NC sur les tables `stg_` pour accélérer les lookups DIM/FACT :
  - `IX_stg_sat_id` sur `stg_airline_satisfaction(ID)` INCLUDE(TypeOfTravel, Class, Satisfaction_Flag, AvgServiceScore)
  - `IX_stg_activity_loyalty` sur `stg_flight_activity(LoyaltyNumber)` INCLUDE(Year, Month, TotalFlights, Distance, PointsAccumulated)
  - `IX_stg_customer_loyalty` sur `stg_customer_loyalty(LoyaltyNumber)` INCLUDE(LoyaltyCard, Country, Province, City, PostalCode)
  - `IX_stg_date_value` sur `stg_date(DateValue)` INCLUDE(StartOfYear, StartOfQuarter, StartOfMonth)
- `UPDATE STATISTICS` sur les 4 tables avec SAMPLE 50 PERCENT

Tâche 11 — `SQL_Validate_Staging`

- `RAISERROR` si une table `stg_` est vide
- `PRINT` le récapitulatif des counts (ex: `Staging OK | Satisfaction:129880 Activity:392936 Customer:16737 Date:2557`)

Package 02 — `02_Populate_Dimensions.dtsx`

Flux :

```
[SQL_DIM_DATE ‖ SQL_DIM_LOYALTY_CARD ‖ SQL_DIM_GEOGRAPHY ‖ SQL_DIM_TRAVEL]
                        |                        |
               (DIM_LOYALTY_CARD et DIM_GEOGRAPHY doivent finir avant)
                                ▼
                          SQL_DIM_CUSTOMER
```

Tâche	Source stg_	Clé de dédup	Colonnes dérivées
`SQL_DIM_DATE`	`stg_date`	`DateValue`	`YEAR()`, `DATEPART(QUARTER)`, `CONCAT('Q',quarter)`, `MONTH()`, `DATENAME(MONTH)`, `DATEPART(WEEKDAY)`, `DATENAME(WEEKDAY)`, `CASE WHEN WEEKDAY IN (1,7) THEN 1 ELSE 0 END`
`SQL_DIM_LOYALTY_CARD`	`stg_customer_loyalty`	`LoyaltyCard`	Aucune
`SQL_DIM_GEOGRAPHY`	`stg_customer_loyalty`	`Country+Province+City+PostalCode`	Aucune
`SQL_DIM_TRAVEL`	`stg_airline_satisfaction`	`TypeOfTravel+Class`	`TravelProfile = CONCAT(TypeOfTravel,' - ',Class)`
`SQL_DIM_CUSTOMER`	`stg_customer_loyalty`	`LoyaltyNumber`	`IsChurned`, `EnrollmentDate`, `SK_LoyaltyCard`, `SK_Geography`, `SCD_StartDate`, `SCD_IsCurrent`

Détail DIM_CUSTOMER (colonnes calculées) :

Colonne	Calcul SQL
`IsChurned`	`CASE WHEN TRY_CAST(CancellationYear AS INT) IS NOT NULL THEN 1 ELSE 0 END`
`EnrollmentDate`	`DATEFROMPARTS(EnrollmentYear, EnrollmentMonth, 1)`
`SK_LoyaltyCard`	`ISNULL((SELECT TOP 1 SK_LoyaltyCard FROM DIM_LOYALTY_CARD WHERE LoyaltyCard=s.LoyaltyCard AND SK_LoyaltyCard>0), -1)`
`SK_Geography`	`ISNULL((SELECT TOP 1 SK_Geography FROM DIM_GEOGRAPHY WHERE Country=s.Country AND Province=s.Province AND City=s.City AND PostalCode=s.PostalCode AND SK_Geography>0), -1)`
`SCD_StartDate`	`GETDATE()`
`SCD_EndDate`	`NULL`
`SCD_IsCurrent`	`1`

Garde NOT EXISTS sur toutes les DIM : `WHERE NOT EXISTS (SELECT 1 FROM DIM_xxx d WHERE d.[NaturalKey]=s.[NaturalKey] AND d.SK_xxx > 0)` — évite les doublons lors de rechargements.

Package 03 — `03_Populate_Facts.dtsx`

Flux : FACT_PASSENGER_SATISFACTION et FACT_FLIGHT_ACTIVITY s'exécutent **en parallèle** (aucune dépendance entre eux).

FACT_PASSENGER_SATISFACTION

Source : `stg_airline_satisfaction`

Colonne FACT	Origine	Calcul
`SK_Date`	—	`-1` (aucune date dans le fichier satisfaction)
`SK_Customer`	—	`-1` (aucun LoyaltyNumber dans le fichier satisfaction)
`SK_Travel`	stg_airline_satisfaction	`ISNULL((SELECT TOP 1 SK_Travel FROM DIM_TRAVEL WHERE TypeOfTravel=s.TypeOfTravel AND Class=s.Class AND SK_Travel>0), -1)`
`SK_Geography`	—	`-1` (aucune géographie dans le fichier satisfaction)
`Gender`, `CustomerType`, `Age`, `FlightDistance`	stg_airline_satisfaction	Copie directe
`DepartureDelay`, `ArrivalDelay`	stg_airline_satisfaction	Copie directe (INT NULL)
`TimeConvenience` … `BaggageHandling`	stg_airline_satisfaction	Copie directe (14 scores INT)
`AvgServiceScore`	stg_airline_satisfaction	Copie directe (DECIMAL(5,2) — calculé en staging)
`Satisfaction`	stg_airline_satisfaction	Copie directe
`Satisfaction_Flag`	stg_airline_satisfaction	Copie directe (BIT — calculé en staging)

FACT_FLIGHT_ACTIVITY

Source : `stg_flight_activity`

Colonne FACT	Origine	Calcul
`SK_Customer`	stg_flight_activity	`ISNULL((SELECT TOP 1 SK_Customer FROM DIM_CUSTOMER WHERE LoyaltyNumber=f.LoyaltyNumber AND SK_Customer>0), -1)`

Colonne FACT	Origine	Calcul
`SK_Date`	stg_flight_activity	`ISNULL((SELECT TOP 1 SK_Date FROM DIM_DATE WHERE DateValue=DATEFROMPARTS(f.Year,f.Month,1) AND SK_Date>0), -1)`
`SK_LoyaltyCard`	stg_flight_activity	`ISNULL((SELECT TOP 1 SK_LoyaltyCard FROM DIM_CUSTOMER WHERE LoyaltyNumber=f.LoyaltyNumber AND SK_Customer>0), -1)`
`SK_Geography`	stg_flight_activity	`ISNULL((SELECT TOP 1 SK_Geography FROM DIM_CUSTOMER WHERE LoyaltyNumber=f.LoyaltyNumber AND SK_Customer>0), -1)`
`TotalFlights`, `Distance`, `PointsAccumulated`, `PointsRedeemed`, `DollarCostPointsRedeemed`	stg_flight_activity	Copie directe

---

Package 04 — `04_Finalize.dtsx`

Flux : `[SQL_Indexes_Fact ‖ SQL_Indexes_Dim]` → `SQL_Update_Stats` → `SQL_Validate_DWH`

Tâche	Action
`SQL_Indexes_Fact`	Crée 5 index NC sur FACT_PASSENGER_SATISFACTION et FACT_FLIGHT_ACTIVITY (FILLFACTOR=90, DATA_COMPRESSION=PAGE, SORT_IN_TEMPDB=ON)
`SQL_Indexes_Dim`	Crée 4 index NC sur DIM_DATE, DIM_CUSTOMER, DIM_GEOGRAPHY, DIM_TRAVEL (FILLFACTOR=90-95)
`SQL_Update_Stats`	`UPDATE STATISTICS` sur les 7 tables DWH avec SAMPLE 50 PERCENT
`SQL_Validate_DWH`	`RAISERROR` si une DIM (SK>0) ou FACT est vide — confirme le chargement complet

---

5. Conception du Data Warehouse

Choix du modèle : Schéma en Étoile (Star Schema)

Justification :

- Requêtes analytiques plus simples et plus performantes (moins de jointures)
- Parfaitement adapté à Power BI (agrégations directes : SUM, AVG, COUNT)
- Lisible et justifiable au jury
- Standard industrie pour les DWH analytiques

Schéma global

```
                        DIM_DATE (SK_Date)
                            │
DIM_GEOGRAPHY ──────────────┤
(SK_Geography)              │
                            ▼
DIM_CUSTOMER ─────── FACT_FLIGHT_ACTIVITY ─────── DIM_LOYALTY_CARD
(SK_Customer)          (table de faits)           (SK_LoyaltyCard)


DIM_TRAVEL ─────── FACT_PASSENGER_SATISFACTION
(SK_Travel)            (table de faits)
```

Définition des tables DIM

Table	Granularité	Clé naturelle	SCD	Lignes
`DIM_DATE`	1 jour	DateValue	Type 0 (statique)	2 557
`DIM_LOYALTY_CARD`	1 niveau de carte	LoyaltyCard	Type 0 (statique)	3
`DIM_GEOGRAPHY`	1 combinaison Country+Province+City+PostalCode	PostalCode+City+Province	Type 1	55
`DIM_CUSTOMER`	1 membre fidélité	LoyaltyNumber	**Type 2**	16 737
`DIM_TRAVEL`	TypeOfTravel × Class	TypeOfTravel+Class	Type 0	6

Définition des tables FACT

Table	Granularité	Lignes	Dimensions liées
`FACT_PASSENGER_SATISFACTION`	1 enquête passager	129 880	DIM_TRAVEL (SK_Travel)
`FACT_FLIGHT_ACTIVITY`	1 membre × 1 mois	392 936	DIM_CUSTOMER, DIM_DATE, DIM_LOYALTY_CARD, DIM_GEOGRAPHY

Justification SCD Type 2 sur DIM_CUSTOMER

Un membre peut upgrader sa carte fidélité (ex: Star → Nova). Sans SCD Type 2, l'historique CLV est perdu. Avec SCD Type 2, chaque changement crée une nouvelle ligne :

```
SK | LoyaltyNumber | LoyaltyCard | SCD_StartDate | SCD_EndDate | SCD_IsCurrent
1  | 100590        | Star        | 2016-01-01    | 2018-06-01  | 0
2  | 100590        | Nova        | 2018-06-01    | NULL        | 1  ← version actuelle
```

Pour toujours cibler la version actuelle : `WHERE SCD_IsCurrent = 1`

Unknown Member Pattern (SK = -1)

Chaque table DIM contient une ligne SK=-1 représentant "valeur inconnue". Toute FACT row dont la clé étrangère ne peut pas être résolue reçoit SK=-1 plutôt que NULL — garantit l'intégrité référentielle.

```
Exemple : FACT_PASSENGER_SATISFACTION.SK_Date = -1
  → aucune date dans le fichier satisfaction (limite documentée)
  → la DIM_DATE contient une ligne SK_Date=-1, DateValue=NULL
```

---

6. Schéma complet des tables DWH

DIM_DATE

```
SK_Date       INT IDENTITY PK
DateValue     DATE
Year          INT
Quarter       INT
QuarterLabel  NVARCHAR(20)   -- 'Q1','Q2','Q3','Q4'
MonthNumber   INT
MonthName     NVARCHAR(20)   -- 'January'...'December'
DayOfWeek     INT
DayName       NVARCHAR(20)   -- 'Monday'...'Sunday'
```

```
IsWeekend     BIT            -- 1 si DATEPART(WEEKDAY) IN (1,7)
StartOfYear   DATE
StartOfQuarter DATE
StartOfMonth  DATE
```

DIM_LOYALTY_CARD

```
SK_LoyaltyCard  INT IDENTITY PK
LoyaltyCard     NVARCHAR(50)   -- 'Star', 'Nova', 'Aurora'
```

DIM_GEOGRAPHY

```
SK_Geography  INT IDENTITY PK
Country       NVARCHAR(100)
Province      NVARCHAR(100)
City          NVARCHAR(100)
PostalCode    NVARCHAR(20)
```

DIM_TRAVEL

```
SK_Travel      INT IDENTITY PK
TypeOfTravel   NVARCHAR(50)   -- 'Business', 'Personal'
Class          NVARCHAR(50)   -- 'Business', 'Economy', 'Economy Plus'
TravelProfile  NVARCHAR(100)  -- 'Business - Business', etc.
```

DIM_CUSTOMER (SCD Type 2)

```
SK_Customer      INT IDENTITY PK
LoyaltyNumber    NVARCHAR(50)   -- clé naturelle (NK)
Gender           NVARCHAR(20)
Education        NVARCHAR(100)
Salary           NVARCHAR(50)   -- conservé en stg, non utilisé en analytique
MaritalStatus    NVARCHAR(50)
CLV              DECIMAL(18,2)
EnrollmentType   NVARCHAR(50)
EnrollmentYear   INT
EnrollmentMonth  INT
CancellationYear INT NULL        -- NULL = membre actif
CancellationMonth INT NULL
IsChurned        BIT             -- 1 si CancellationYear IS NOT NULL
SK_LoyaltyCard   INT FK → DIM_LOYALTY_CARD
SK_Geography     INT FK → DIM_GEOGRAPHY
EnrollmentDate   DATE            -- DATEFROMPARTS(EnrollmentYear, EnrollmentMonth, 1)
SCD_StartDate    DATE
SCD_EndDate      DATE NULL
SCD_IsCurrent    BIT DEFAULT 1
```

FACT_PASSENGER_SATISFACTION

```
SK_Satisfaction  INT IDENTITY PK
SK_Date          INT DEFAULT -1 FK → DIM_DATE       -- toujours -1 (aucune date source)
SK_Customer      INT DEFAULT -1 FK → DIM_CUSTOMER   -- toujours -1 (aucun LoyaltyNumber source)
SK_Travel        INT DEFAULT -1 FK → DIM_TRAVEL     -- résolu via TypeOfTravel+Class
SK_Geography     INT DEFAULT -1 FK → DIM_GEOGRAPHY  -- toujours -1 (aucune géo source)
```

```
Gender           NVARCHAR(20)
CustomerType     NVARCHAR(50)
Age              INT
FlightDistance   INT
DepartureDelay   INT NULL       -- NULL si vide dans CSV
ArrivalDelay     INT NULL       -- NULL si vide dans CSV
TimeConvenience  INT            -- 0-5
OnlineBooking    INT
CheckinService   INT
OnlineBoarding   INT
GateLocation     INT
OnboardService   INT
SeatComfort      INT
LegRoom          INT
Cleanliness      INT
FoodAndDrink     INT
InFlightService  INT
InFlightWifi     INT
InFlightEntertainment INT
BaggageHandling  INT
AvgServiceScore  DECIMAL(5,2)  -- (somme 14 scores) / 14.0
Satisfaction     NVARCHAR(50)
Satisfaction_Flag BIT          -- 1=Satisfied, 0=Neutral or Dissatisfied
```

FACT_FLIGHT_ACTIVITY

```
SK_Activity                INT IDENTITY PK
SK_Customer                INT DEFAULT -1 FK → DIM_CUSTOMER
SK_Date                    INT DEFAULT -1 FK → DIM_DATE
SK_LoyaltyCard             INT DEFAULT -1 FK → DIM_LOYALTY_CARD
SK_Geography               INT DEFAULT -1 FK → DIM_GEOGRAPHY
TotalFlights               INT
Distance                   DECIMAL(18,2)
PointsAccumulated          DECIMAL(18,2)
PointsRedeemed             DECIMAL(18,2)
DollarCostPointsRedeemed   DECIMAL(18,2)
```

---

7. KPIs — Requêtes analytiques de référence

KPI-1 — Satisfaction Rate global

```sql
USE DWH_DB;
SELECT
    ROUND(100.0 * SUM(CAST(Satisfaction_Flag AS INT)) / COUNT(*), 2) AS SatisfactionRate_Pct,
    COUNT(*) AS TotalSurveys,
    SUM(CAST(Satisfaction_Flag AS INT)) AS Satisfied
FROM dbo.FACT_PASSENGER_SATISFACTION;
-- Cible : 73.1%
```

KPI-2 — Satisfaction par classe de voyage

```sql
SELECT t.TypeOfTravel, t.Class,
    ROUND(100.0 * SUM(CAST(f.Satisfaction_Flag AS INT)) / COUNT(*), 2) AS SatisfactionRate_Pct,
    COUNT(*) AS Total
FROM dbo.FACT_PASSENGER_SATISFACTION f
JOIN dbo.DIM_TRAVEL t ON f.SK_Travel = t.SK_Travel AND t.SK_Travel > 0
```

```
GROUP BY t.TypeOfTravel, t.Class
ORDER BY SatisfactionRate_Pct DESC;
```

KPI-3 — Score moyen par service (identifier les < 3.5)

```
SELECT service_name, ROUND(AVG(CAST(score AS DECIMAL(5,2))), 2) AS AvgScore
FROM (
    SELECT 'Time Convenience'       service_name, TimeConvenience       score FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Online Booking',                      OnlineBooking              FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Check-in Service',                    CheckinService             FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Online Boarding',                     OnlineBoarding             FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Gate Location',                       GateLocation               FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'On-board Service',                    OnboardService             FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Seat Comfort',                        SeatComfort                FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Leg Room',                            LegRoom                    FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Cleanliness',                         Cleanliness                FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Food and Drink',                      FoodAndDrink               FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'In-flight Service',                   InFlightService            FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'In-flight Wifi',                      InFlightWifi               FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'In-flight Entertainment',             InFlightEntertainment      FROM
dbo.FACT_PASSENGER_SATISFACTION UNION ALL
    SELECT 'Baggage Handling',                    BaggageHandling            FROM
dbo.FACT_PASSENGER_SATISFACTION
) s
WHERE score > 0   -- 0 = service non applicable, exclu des moyennes
GROUP BY service_name
ORDER BY AvgScore ASC;
```

KPI-4 — Impact des retards sur la satisfaction

```
SELECT
    CASE
        WHEN ISNULL(DepartureDelay,0) = 0            THEN '0 — Aucun retard'
        WHEN ISNULL(DepartureDelay,0) BETWEEN 1 AND 30   THEN '1-30 min'
        WHEN ISNULL(DepartureDelay,0) BETWEEN 31 AND 60  THEN '31-60 min'
        ELSE '> 60 min'
    END AS DelayBucket,
    COUNT(*) AS Total,
    ROUND(100.0 * SUM(CAST(Satisfaction_Flag AS INT)) / COUNT(*), 2) AS SatisfactionRate_Pct
FROM dbo.FACT_PASSENGER_SATISFACTION
GROUP BY
    CASE
        WHEN ISNULL(DepartureDelay,0) = 0            THEN '0 — Aucun retard'
        WHEN ISNULL(DepartureDelay,0) BETWEEN 1 AND 30   THEN '1-30 min'
        WHEN ISNULL(DepartureDelay,0) BETWEEN 31 AND 60  THEN '31-60 min'
        ELSE '> 60 min'
    END
ORDER BY MIN(ISNULL(DepartureDelay,0));
```

KPI-5 — Churn Rate par type de carte

```sql
SELECT lc.LoyaltyCard,
    COUNT(*) AS TotalMembers,
    SUM(CAST(c.IsChurned AS INT)) AS Churned,
    ROUND(100.0 * SUM(CAST(c.IsChurned AS INT)) / COUNT(*), 2) AS ChurnRate_Pct,
    ROUND(100.0 * (1 - 1.0*SUM(CAST(c.IsChurned AS INT)) / COUNT(*)), 2) AS RetentionRate_Pct
FROM dbo.DIM_CUSTOMER c
JOIN dbo.DIM_LOYALTY_CARD lc ON c.SK_LoyaltyCard = lc.SK_LoyaltyCard
WHERE c.SK_Customer > 0
GROUP BY lc.LoyaltyCard
ORDER BY ChurnRate_Pct DESC;
-- Cible churn : ≤ 60%
```

KPI-6 — CLV moyen par type de carte

```sql
SELECT lc.LoyaltyCard,
    COUNT(*) AS Members,
    ROUND(AVG(c.CLV), 2) AS AvgCLV,
    ROUND(MIN(c.CLV), 2) AS MinCLV,
    ROUND(MAX(c.CLV), 2) AS MaxCLV
FROM dbo.DIM_CUSTOMER c
JOIN dbo.DIM_LOYALTY_CARD lc ON c.SK_LoyaltyCard = lc.SK_LoyaltyCard
WHERE c.SK_Customer > 0 AND c.CLV IS NOT NULL
GROUP BY lc.LoyaltyCard
ORDER BY AvgCLV DESC;
```

KPI-7 — Taux de rédemption des points

```sql
SELECT lc.LoyaltyCard,
    SUM(f.PointsAccumulated) AS TotalAccumulated,
    SUM(f.PointsRedeemed) AS TotalRedeemed,
    ROUND(100.0 * SUM(f.PointsRedeemed) / NULLIF(SUM(f.PointsAccumulated), 0), 2) AS
RedemptionRate_Pct
FROM dbo.FACT_FLIGHT_ACTIVITY f
JOIN dbo.DIM_CUSTOMER c ON f.SK_Customer = c.SK_Customer
JOIN dbo.DIM_LOYALTY_CARD lc ON f.SK_LoyaltyCard = lc.SK_LoyaltyCard
WHERE f.SK_Customer > 0 AND f.SK_LoyaltyCard > 0
GROUP BY lc.LoyaltyCard;
```

KPI-8 — NPS Proxy par classe de voyage

```sql
SELECT t.TypeOfTravel, t.Class,
    SUM(CAST(Satisfaction_Flag AS INT)) AS Promoters,
    SUM(CASE WHEN Satisfaction_Flag = 0 THEN 1 ELSE 0 END) AS Detractors,
    SUM(CAST(Satisfaction_Flag AS INT)) - SUM(CASE WHEN Satisfaction_Flag = 0 THEN 1 ELSE 0 END) AS
NPS_Proxy
FROM dbo.FACT_PASSENGER_SATISFACTION f
JOIN dbo.DIM_TRAVEL t ON f.SK_Travel = t.SK_Travel AND t.SK_Travel > 0
GROUP BY t.TypeOfTravel, t.Class
ORDER BY NPS_Proxy DESC;
-- Valeur positive attendue
```

---

8. Points critiques pour la validation jury

Ce que le jury va vérifier

Critère	Attendu	Statut
Modèle de données	Schéma étoile clairement dessiné avec FK	✅ 5 DIM + 2 FACT
Justification modèle	Pourquoi étoile ? (performance, simplicité BI)	✅ Documenté
Granularité	Clairement définie pour chaque fait	✅ Documentée
SCD	Type 2 sur DIM_CUSTOMER justifié avec exemple	✅ Implémenté
Nommage SK_	Clés de substitution préfixées	✅ SK_Customer, SK_Date...
ETL — outil	SSIS (SQL Server Integration Services)	✅ 5 packages + Master
ETL — phases	Extract → Transform → Load documentés	✅ Sections 3+4
Qualité données	Problèmes identifiés + solutions appliquées	✅ Section 2
KPIs fonctionnels	Cohérence avec objectifs (57% → 73.1%)	✅ 8 requêtes
Unknown Member	SK=-1 dans chaque DIM	✅ Implémenté
Index + stats	Optimisation post-chargement	✅ Package 04

Erreurs à éviter absolument

1. **Ne pas confondre** table de faits et dimension (les mesures vont dans les faits)
2. **Ne pas oublier** les SK_ — jamais les clés métier directement dans les faits
3. **Les scores** `0` ne sont pas des "notes basses" — c'est "service non applicable" (à expliquer au jury)
4. **L'absence de lien** entre Loyalty et Satisfaction est une limite documentée, pas une erreur
5. **Dates toujours** en type DATE ou INT (YYYYMMDD) — jamais VARCHAR
6. **La granularité** est la première question du jury — savoir répondre en une phrase
7. **SCD Type 2** : expliquer que l'historique n'a pas encore été exercé (chargement initial, tous `SCD_IsCurrent=1`)

Questions jury anticipées

Question	Réponse
Quelle est la granularité de FACT_PASSENGER_SATISFACTION ?	1 ligne = 1 enquête de satisfaction par passager
Quelle est la granularité de FACT_FLIGHT_ACTIVITY ?	1 ligne = 1 membre fidélité × 1 mois
Pourquoi SK_Date=-1 dans FACT_PASSENGER_SATISFACTION ?	Le fichier satisfaction n'a pas de date — limite documentée de la source
Pourquoi pas de lien entre les deux tables de faits ?	Les deux sources n'ont aucune clé commune
Qu'est-ce que TRY_CAST ?	Conversion de type qui retourne NULL en cas d'échec, sans arrêter le pipeline
Pourquoi les scores 0 sont conservés ?	0 = service non applicable (ex: Wifi sur vol court-courrier) — exclus des moyennes avec WHERE score > 0

Projet Airline Analytics DWH — Esprit Engineering, 3ème année — 2026-04-24